

## **1. SCENARIO 1: UNIVERSITY STUDENT MANAGEMENT SYSTEM**

A university manages information related to students, faculty members, departments, courses, and examinations. Currently, student information is stored in spreadsheets by different departments, resulting in duplicated records, inconsistent student IDs, and difficulties in generating reports. The university administration wants to develop a centralized database system that can store student details, course enrollments, faculty assignments, attendance records, and examination results. Students should be able to register for multiple courses, while each course may have multiple students enrolled. Faculty members may teach several courses within a semester. The management also requires reports such as department-wise student strength, faculty workload, and student performance statistics. Design an appropriate database schema by identifying entities, attributes, primary keys, foreign keys, and relationships. Connect the database using Python and demonstrate CRUD operations for managing student records.

### **SOLUTION:**

#### **CODE:**

```
import sqlite3

conn = sqlite3.connect("University.db")

cursor = conn.cursor()

cursor.execute("""

CREATE TABLE IF NOT EXISTS Student(

    Student_ID INTEGER PRIMARY KEY,

    Student_Name TEXT NOT NULL,

    Gender TEXT,

    Department TEXT,

    Email TEXT

)

""")

conn.commit()

cursor.execute("DELETE FROM Student")

conn.commit()

print("CREATE OPERATION")
```

```
cursor.execute("""
INSERT INTO Student
(Student_ID, Student_Name, Gender, Department, Email)
VALUES
(101, 'Rahul', 'Male', 'AI&DS', 'rahul@gmail.com')
""")

conn.commit()

print("Student Record Inserted Successfully.\n")

print("READ OPERATION")

cursor.execute("SELECT * FROM Student")

rows = cursor.fetchall()

for row in rows:

    print(row)

print()

print("UPDATE OPERATION")

cursor.execute("""
UPDATE Student
SET Department='CSE'
WHERE Student_ID=101
""")

conn.commit()

print("Student Record Updated.\n")

cursor.execute("SELECT * FROM Student")

for row in cursor.fetchall():

    print(row)

print()

print("DELETE OPERATION")

cursor.execute("""
DELETE FROM Student
WHERE Student_ID=101
""")

conn.commit()

print("Student Record Deleted.\n")
```

```
cursor.execute("SELECT * FROM Student")
```

```
rows = cursor.fetchall()
```

```
if len(rows) == 0:
```

```
    print("No Records Found")
```

```
else:
```

```
    for row in rows:
```

```
        print(row)
```

```
conn.close()
```

## **OUTPUT:**

CREATE OPERATION

Student Record Inserted Successfully.

READ OPERATION

(101, 'Rahul', 'Male', 'AI&DS', 'rahul@gmail.com')

UPDATE OPERATION

Student Record Updated.

(101, 'Rahul', 'Male', 'CSE', 'rahul@gmail.com')

DELETE OPERATION

Student Record Deleted.

No Records Found

## 2. SCENARIO 6: BANKING AND CUSTOMER ACCOUNT MANAGEMENT SYSTEM

A commercial bank manages savings accounts, current accounts, loans, and customer transactions. Customers may own multiple accounts, and each account contains transaction records such as deposits, withdrawals, and transfers. The bank requires a secure database system to store customer information, account details, branch information, and loan records. Managers need reports on customer balances, transaction histories, and loan repayments. The system must support quick retrieval of information and ensure data consistency. Design the database schema, define relationships between customers and accounts, and implement Python-based database connectivity for account management operations.

### SOLUTION:

#### CODE:

```
import sqlite3

conn = sqlite3.connect("BankingSystem.db")

cursor = conn.cursor()

cursor.execute("""

CREATE TABLE IF NOT EXISTS Account(

    Account_No INTEGER PRIMARY KEY,

    Customer_Name TEXT NOT NULL,

    Account_Type TEXT,

    Branch_Name TEXT,

    Balance REAL

)

""")

conn.commit()

cursor.execute("DELETE FROM Account")

conn.commit()

print("CREATE OPERATION ")

cursor.execute("""

INSERT INTO Account

(Account_No, Customer_Name, Account_Type, Branch_Name, Balance)

VALUES

(1001, 'Rahul', 'Savings', 'Chennai Branch', 50000)

""")
```

```
conn.commit()

print("Account Created Successfully.\n")

print(" READ OPERATION ")

cursor.execute("SELECT * FROM Account")

records = cursor.fetchall()

for row in records:

    print(row)

print("\n UPDATE OPERATION ")

cursor.execute("""

UPDATE Account

SET Balance = 65000

WHERE Account_No = 1001

""")

conn.commit()

print("Account Balance Updated Successfully.\n")

cursor.execute("SELECT * FROM Account")

for row in cursor.fetchall():

    print(row)

print("\nDELETE OPERATION ")

cursor.execute("""

DELETE FROM Account

WHERE Account_No = 1001

""")

conn.commit()

print("Account Deleted Successfully.\n")

cursor.execute("SELECT * FROM Account")

records = cursor.fetchall()

if records:

    for row in records:

        print(row)

else:

    print("No Records Found")

conn.close()
```

## **OUTPUT:**

### **CREATE OPERATION**

Account Created Successfully.

### **READ OPERATION**

(1001, 'Rahul', 'Savings', 'Chennai Branch', 50000.0)

### **UPDATE OPERATION**

Account Balance Updated Successfully.

(1001, 'Rahul', 'Savings', 'Chennai Branch', 65000.0)

### **DELETE OPERATION**

Account Deleted Successfully.

No Records Found